JARVIS / AURON — Canonical Master Plan

Status: canonical project roadmap
Repository: `ScalpingEdward/JARVIS_os`

1. Mission

JARVIS is the master operating system. AURON is its controlled intelligence/orchestration layer. Vertical capabilities operate through one governed core, one Command Centre, shared identity/audit state, and explicit safety boundaries. Build sequence must be preserved.

2. Source-of-truth rule for every new chat/build session

1. Read this file from `main`.
2. Verify the last claimed PR is actually merged into `main`.
3. Inspect current implementation on `main`.
4. Continue from the checkpoint and dependency graph.
5. Never restart merged layers or invent merge state.
6. Reconcile implementation and roadmap explicitly if they disagree.

3. Architecture invariants

- JARVIS = master OS; AURON = orchestration/intelligence/governance.
- Command Centre remains operational UI with persistent command/text interaction.
- Consequential external actions require explicit execution boundaries, observable results, audit and failure state.
- Simulation remains independent of external execution.
- Provider adapters cannot bypass central policy/risk/approval controls.
- Idempotency, reconciliation, health, provenance/version state and fail-closed behavior are core infrastructure.
- Secrets never belong in source control.
- External execution is deliberate, never default.

4. Completed foundation lineage

Foundation successor governance closed at v21.523. Phase A core cutover is complete through A6.

5. Mandatory build sequence

Phase B — Trading (reconciled)

The real, wired path is `backend/app/trading/api.py` -> `service.py` (`GET /v1/trading/status`, `POST /v1/trading/evaluate`, setup listing) registered in `main.py`. This is genuinely usable today.

Fourteen `auron_*_v21_53x` / `v21_604-607` modules in `backend/app/trading` and `backend/app/core` (registry, account state, signal intake, risk engine, multi-account allocation, guards, MT5 adapter, reconciliation canary, controlled live enablement, command centre, shadow-canary selection/adapter/

certification/health) plus their `api/routes` command-centre file were audited the same way as the Research H21-H40 chain: never imported by `main.py`, never called by `trading/api.py` or `trading/service.py`, only importing and testing each other. They have been removed. Full suite green afterward (4615 passed).

Note preserved during audit: `backend/app/core/auron_integration_readiness_vNNN.py` files form a sequential linked chain (each version imports the previous one, `v21_539` -> `v21_538` -> ... -> `v21_1`) shared across every vertical, not a per-vertical ledger. Deleting a file in the middle of this chain breaks every higher version that has not itself been reconciled/removed yet. Do not delete an `auron_integration_readiness_vNNN.py` file unless every version above it in the chain has already been removed or repointed. This chain itself is a candidate for a future, carefully-sequenced cleanup, not an ad hoc deletion.

B1-B10 as a phase label is retired in favor of the concrete statement above: one real endpoint set exists and works; the rest was ceremony.

Phase C — Instagram Content Manager

Completed through C8; provider writes remain gated.

Phase D — Additional verticals

Communications D1-D8, Research D9-D16, Automation D17-D24 and Files & Documents D25-D32 completed.

Phase E — Cross-vertical integration certification

E1-E4 completed.

Phase F — Controlled provider canary program

F1-F4 completed.

Phase G — Provider-specific controlled canary integration

Research G1-G4, Instagram G5-G8, Files & Documents G9-G12, Communications G13-G16 and Trading-shadow G17-G20 are complete. G21 completes the evidence-driven expansion decision.

Phase H — Controlled external-provider sandbox integration (reconciled)

H1-H20 established a real, contract-bound, read-only sandbox adapter (no network transport, no credential resolution) and are retained.

H21-H40 (transport injection, authorization gates, execution-boundary design, and the H40 request-execution design contract) were audited against the actual repository and found to be fully disconnected: none of the ~50 modules in this range were imported by `main.py` or any other application code, they only imported and tested each other, and the one concrete “request” they modeled targeted a non-existent `sandbox.example.test` domain. No module in this range ever performed, or could perform, a network call. This range (32 research modules + 32 paired `core` integration-readiness modules + their tests, ~96 files) has been removed rather than carried forward, since it added no real capability and no real safety guarantee beyond what H1-H20 already provide.

Replacement: `backend/app/research/real_provider_client.py` + `backend/app/research/provider_api.py`, wired into `main.py` at `POST /v1/research/provider/fetch`. This is a real, executable client: HTTPS-only, explicit domain allowlist (currently `api.github.com`), bounded timeout, bounded response size, exactly-once per `request_id` via an append-only sqlite audit log, no write capability, no

credential handling. Covered by `backend/tests/test_research_real_provider_client.py` (allowlist rejection, scheme rejection, size-limit enforcement, timeout handling, exactly-once, and the wired API endpoint), all passing alongside the full existing suite (4694 tests green after this change).

Phase H is considered functionally complete for a first real Research provider path. Extending the allowlist to additional domains/providers is a deliberate, reviewed code change to `DEFAULT_ALLOWED_DOMAINS`, never a runtime parameter.

Lesson carried forward: future phases must add one real, wired, testable capability per PR rather than a new design/certification layer describing a capability that is not yet wired in. If a PR's diff does not touch `main.py`'s router registration or an equivalent real entrypoint, it does not ship user-visible functionality and should say so explicitly rather than advancing the phase counter.

Reconciliation (2026-08-30) — physical dead-code removal of `backend/app/research/auron_research_*`: the roadmap above described the H21-H40 chain as "removed", but a fresh import-graph audit from `backend/app/main.py` found that 19 `auron_research_*_v21_5xx/6xx.py` modules were still physically present in `backend/app/research/` and fully unreachable. `main.py` wires only `research/provider_api.py` (+ `research/real_provider_client.py`) via `POST /v1/research/provider/fetch`; none of the 19 modules were imported by `main.py` or by any other reachable application module — they imported and tested only each other (a closed, self-referential cluster). All 19 modules and their 19 paired `backend/tests/test_auron_research_*` files were deleted in this PR. The full suite is green afterwards (4515 passed, 0 failed; 100 tests belonging to the removed cluster dropped from the prior 4615). No `backend/app/core/auron_integration_readiness_vNNN.py` file was touched — the sequential readiness chain (93 files) is preserved intact per the note above. Removed research modules: `adapter_onboarding_v21_556`, `registry_evidence_v21_557`, `read_search_fetch_v21_558`, `evidence_policy_v21_559`, `report_simulation_v21_560`, `controlled_watch_v21_561`, `watch_reconciliation_v21_562`, `command_centre_v21_563`, `readonly_canary_adapter_v21_588`, `canary_e2e_certification_v21_589`, `provider_health_drift_v21_590`, `canary_command_centre_v21_591`, `external_readonly_sandbox_adapter_v21_610`, `external_sandbox_e2e_reconciliation_v21_611`, `external_sandbox_health_drift_observability_v21_612`, `network_transport_authorization_v21_613`, `readonly_network_transport_boundary_v21_614`, `readonly_network_boundary_e2e_certification_v21_615`, `real_provider_activation_readiness_v21_616`.

6. Cross-vertical Command Centre requirements

Persistent command/text interaction, capability health, simulation/execution visibility, approvals, execution timeline/failures, dedicated vertical workspaces, kill switches and audit/evidence access remain mandatory.

7. Definition of usable

A vertical requires observable provider health, persistent state, policy before outbound action, idempotency/reconciliation, visible results, disable controls, failure/replay tests and deliberate external enablement.

8. PR discipline

One coherent layer per PR, with tests, dependencies and next layer documented. Verify merge before advancing.

9. Current checkpoint

- Foundation through v21.523 and Phase A A1-A6: complete.
- Trading B1-B10, Instagram C1-C8 and Phase D vertical architecture: complete with consequential provider execution gated.
- Phase E E1-E4, Phase F F1-F4 and Phase G through G21: complete.
- H1-H20 complete: Research provider path is contract-bound, reconciled, certified through one bounded transport-disabled session and zero-concrete-transport injection contract.
- H21-H31 complete: governed transport activation, authorization, binding, injection design/certification and exactly one inert read-only transport object are established and certified with network execution disabled.
- H32-H35 complete: one short-lived operator-bound revocable one-shot network-execution authorization is designed, certified, issued and certified again against exact transport/object/identity scope while provider traffic remains disabled.
- H36-H37 complete: the fail-closed exactly-once consumption boundary is designed and certified with expiry/revocation/read-only scope and zero network state.
- H21-H40 (32 research modules + 32 paired core modules + tests) removed: confirmed disconnected from `main.py` and from each other's non-test callers, tested only against a non-existent sandbox domain, and never capable of a real network call. Removal verified against the full test suite (4694 passed, 0 failed).
- **Research provider path is now real and wired**: `POST /v1/research/provider/fetch` performs an actual bounded HTTPS GET against an explicit domain allowlist, with timeout, response-size limit, exactly-once request handling and an append-only audit log. This supersedes the H21-H40 design chain.
- **Dead-code reconciliation (2026-08-30):** 19 unreachable `backend/app/research/auron_research_*` modules that the roadmap had already described as removed but were still on disk, plus their 19 paired `backend/tests/test_auron_research_*` files, have now been physically deleted (branch `chore/remove-dead-auron-research-modules`). Verified unreachable via an import-graph audit from `backend/app/main.py`; no `auron_integration_readiness_vNNN.py` file touched. Full suite green: 4515 passed, 0 failed (down from 4615 as the 100 tests of the removed cluster were deleted).
- **Multi-vertical dead-code reconciliation (2026-08-30):** A comprehensive import-graph audit from `backend/app/main.py` identified 47 additional unreachable `auron_*` modules across multiple verticals (8 automation, 11 communications, 3 content/instagram, 14 core, 11 documents), all confirmed disconnected from the live application. These modules, plus their 47 paired test files (94 files total), have been physically deleted (branch `chore/remove-dead-auron-modules-multi-vertical`). No `auron_integration_readiness_vNNN.py` file touched — the sequential readiness chain remains intact. Full suite green: 4245 passed, 0 failed (down from 4515 as 270 tests of the removed clusters were deleted).
- Next: extend the Research allowlist deliberately as real needs arise; consider real wiring for Trading, Instagram, Communications, Automation, and Documents verticals following the Research pattern (one real endpoint, explicit safety boundaries, audit log).

This checkpoint must be updated at each phase boundary or major activation milestone.